Nakijk 03_02 — V2

NAKIJK V2 · Hoofdstuk 03 · CRUD via een repository · bij de nieuwe versie van Oefening_03_02.md · 8 november 2026

Nagekeken: je project in `Oefeningen Hoofdstuk 3`, zoals het vanochtend om 06:21 bewaard was. Ook nu is er hier geen .NET: ik heb je code gelezen en elk verzoek op papier nagelopen.

De opgave is tegelijk herschreven. Vervangen staat nu stap voor stap in punt 4, en punt 6 is opgedeeld in vijf delen, 6a tot 6e, met een voorbeeld van de volledige historiek. Wat je API moet antwoorden, is niet veranderd. De vorige opgave staat in de submap `Vorige versie`. De stappen in deze nakijk verwijzen naar de nieuwe versie.

Legende: ✓ in orde · ✗ fout of afwijking, met de plaats waar je het oplost · ○ nog niet gemaakt

Kort oordeel

Je hebt de grootste fout uit V1 opgelost: een PUT laat de Id nu met rust. Je Warnings vertellen de echte reden, en elk endpoint logt een Information-bericht. Maar je project bouwt op dit moment niet, door twee onafgewerkte regels in je wijzigingsrepository (A1). Zolang dat zo is, draait er geen enkele nieuwe regel van je code. Daarnaast laat PUT een prijs van 0 door (A2).

Volgorde: eerst A1, dan A2, B1 en B2. Lees daarna *Je vraag over punt 5*, en volg dan de twee stappenplannen: eerst vervangen, dan de historiek. Vervangen eerst, omdat je in Historiek 8 ook in je vervang-endpoint wijzigingen bijschrijft.

Takenlijst

1. A1 — je project bouwt niet (`InMemoryWijzigingRepository`, regel 8 en 9)
2. A2 — PUT laat een prijs van 0 door (controller, regel 88)
3. B1 — het Information-bericht komt pas na de controles
4. B2 — `VervangVentilator` uit het contract halen
5. Je vraag over punt 5 — wat bij vervangen hoort, en wat niet
6. Stappenplan vervangen — Vervangen 1 tot 7
7. Stappenplan historiek — Historiek 1 tot 9, en het testscenario
8. C — opkuis, zonder invloed op het antwoord

Stand van zaken

Zoals het zal zijn zodra A1 opgelost is. Nu bouwt je project niet, en antwoordt je API dus niet.

Punt	Wat de opgave vraagt	Status
1	de zes endpoints antwoorden zoals in 02_02	✓ GET, GET op Id, POST, DELETE ✓ PUT: de Id-fout uit V1 is opgelost ✗ PUT met prijs 0: A2 ○ vervangen: staat in commentaar
2	het contract `IVentilatorRepository`	✓ de vijf methodes ✗ één te veel: B2
3	de repository, die zelf de Id bepaalt	✓ in orde
4	wie beslist wat, de constructor, Program.cs en de logging	✓ beslissingen en injectie ✓ de reden in elke Warning (B2 uit V1 opgelost) ✗ Information pas na de controles: B1
5	gegevens tussen twee verzoeken	✓ je ventilatoren ✗ je wijzigingen: Historiek 3

Punt	Wat de opgave vraagt	Status
6	de wijzigingshistoriek	○ het model ✓, het contract half, de repository bouwt niet (A1), de rest nog te doen: stappenplan

Zo antwoordt je API nu: niet. Druk je in Visual Studio toch op Start, dan vraagt het *There were build errors. Would you like to continue and run the last successful build?* Kies **No**. Met **Yes** test je een oudere versie van je code, en lijkt het alsof je aanpassingen niets doen.

Zodra A1 opgelost is, antwoordt je API zo, vanaf een herstart:

Verzoek	Je API zal antwoorden	De opgave vraagt
GET `api/ventilator`	200 met je vier ventilatoren	✓
POST `api/ventilator`, 92 mm	400 met *We verkopen enkel ventilatoren van 120 of 140 mm.*	✓ (in je log geen `info:` ervoor: B1)
PUT `api/ventilator/2`, body zonder `id`	204, en ventilator 2 houdt Id 2	✓ opgelost sinds V1
PUT `api/ventilator/2`, prijs 0	204, en ventilator 2 kost daarna 0	400 met *De prijs moet groter zijn dan 0.* (A2)
DELETE `api/ventilator/4`, twee keer	204, daarna 404 met het bericht	✓
PUT `api/ventilator/vervang/3`	404 zonder body: het endpoint staat in commentaar	201 met de nieuwe ventilator ○
GET `api/ventilator/historiek`	400: komt bij GET `{id}` terecht	200 met `[]` ○

Wat goed zit, en waarom

Bijwerken laat de Id met rust (`InMemoryVentilatorRepository`, regel 32 tot 43). Je neemt enkel de vier gegevens over die de client mag veranderen. De URL zegt welke ventilator, de body wat er verandert. Dat heb je straks ook nodig: bij `bijgewerkt` hoort in de historiek de Id uit de URL, en die klopt nu ook na de PUT nog.

Je Warnings vertellen de echte reden (regel 51, 57 en 84). Wie later je log leest, weet meteen waarom een verzoek geweigerd werd, zonder je code te openen.

Je begon vervangen in de controller. Daar hoort het endpoint ook: de controller beslist, de repository voert uit. Enkel de methode die je er in de repository bij maakte, is overbodig (B2).

Je lijst met ventilatoren is nog altijd `static`, en `Create` geeft het object terug waarmee `CreatedAtAction` het adres bouwt. Op die twee bouw je vervangen en de historiek verder.

A. Wat een verkeerd antwoord geeft

A1. Je project bouwt niet ✗

Waar: `InMemoryWijzigingRepository.cs`, regel 8 en 9.

Regel 8 bevat enkel het woord `Wijziging`: een regel die nooit afgemaakt werd. Regel 9 gebruikt een naam `wijzi` die nergens bestaat, en geeft een lambda mee aan `OrderDescending`, een methode die geen lambda verwacht. In de Foutenlijst (Error List) van Visual Studio staan daarom rode fouten voor beide regels, onder meer *The name 'wijzi' does not exist in the current context*.

Waarom het hele project stilvalt. C# bouwt je project als één geheel. Eén bestand met een fout, en er komt geen nieuwe versie van je API. Ook wat je in je controller veranderde, draait dan niet.

Wat de opgave hier vraagt (deel 6b): alle wijzigingen, de oudste eerst. Drie dingen aan regel 9 kloppen daar niet mee:

- *Descending* betekent aflopend: de grootste eerst, dus de nieuwste. Dat is het omgekeerde van wat gevraagd is.
- `OrderDescending` vergelijkt de elementen zelf met elkaar. Een `Wijziging` weet niet hoe ze zich met een andere vergelijkt, dus ook zonder lambda zou het misgaan, pas als je API draait. Op één gegeven sorteren doet `OrderByDescending`. Je leerboek zet ze naast elkaar in hoofdstuk 11.
- Sorteren geeft geen `List` terug, maar een reeks (hoofdstuk 12). Je methode belooft een `List`.

Richting. Je hoeft hier niet te sorteren. Een `List` zet wat je toevoegt altijd achteraan (hoofdstuk 6). De volgorde in je lijst is dus al de volgorde waarin de wijzigingen gebeurden. Kijk hoe `GetAll` in je ventilatorrepository zijn lijst teruggeeft.

Hoe je weet dat het opgelost is. Kies *Build* en dan *Rebuild Solution*. In het venster Output staat *1 succeeded, 0 failed*, en de Foutenlijst toont 0 fouten.

A2. PUT laat een prijs van 0 door ✗

Waar: `VentilatorController.cs`, regel 88.

Je POST weigert een prijs van 0 of lager (regel 55). Je PUT weigert enkel een prijs onder 0. Een PUT met prijs 0 geeft daardoor 204, en de ventilator kost daarna niets. Je Warning op regel 90 zegt nochtans *niet onder 0 of 0*: het bericht klopt met de regel, de voorwaarde niet.

Waarom zoiets gebeurt. Je schrijft dezelfde drie controles twee keer, en straks bij vervangen een derde keer. Elke kopie kan apart afwijken, en het nakijkscript test de prijs enkel bij een POST. Wil je dat vermijden, dan kan je de drie controles op één plaats in de controller zetten en die plaats vanuit de drie endpoints gebruiken. Controleren blijft controllerwerk: het hoort niet in de repository. Dat is een keuze, geen eis.

Hoe je weet dat het opgelost is. PUT `api/ventilator/2` met prijs 0 geeft 400 met *De prijs moet groter zijn dan 0.*, en GET `api/ventilator/2` toont daarna nog de oude prijs.

B. Werkt, maar wijkt af van de opgave

B1. Het Information-bericht komt pas na de controles ✗

Waar: `VentilatorController.cs`, regel 62, 94 en 112.

Sinds V1 logt elk endpoint een Information-bericht ✓. Maar in POST, PUT en DELETE staat het na de controles, en in POST en DELETE zelfs na de actie. Een verzoek dat geweigerd wordt, komt er nooit langs. In je log staat dan een `warn:` zonder `info:` ervoor, alsof de Warning uit het niets kwam.

Punt 4 vraagt *elk binnenkomend verzoek*, dus ook de geweigerde. De nieuwe opgave zegt het er nu bij: als eerste regel van het endpoint.

Richting. Kijk naar je twee GET's: daar staat het al goed. Pas ook de tekst aan: op de eerste regel is er nog niets aangepast of verwijderd. Regel 94 zegt nu al *aangepast*, nog vóór `Update`.

Hoe je het controleert. POST een ventilator van 92 mm en kijk in je terminal: eerst een regel met `info:`, dan een met `warn:`.

B2. `VervangVentilator` staat in het contract, maar doet niets ✗

Waar: `IVentilatorRepository.cs`, regel 10, en `InMemoryVentilatorRepository.cs`, regel 55 tot 58.

In V1 heette deze methode `Replace`. De naam is veranderd, het probleem niet. Punt 2 somt vijf dingen op, en de nieuwe opgave zegt het nu uitdrukkelijk in punt 4: vervangen krijgt geen eigen methode. Drie redenen:

- Tussen het opzoeken en het uitvoeren moet de controller de drie controles doen. Dat kan de repository niet: ze beslist niets.
- De historiek schrijft bij vervangen twee wijzigingen, met twee verschillende Id's. Die bewaart de controller, dus hij moet beide Id's kennen.
- De methode krijgt de nieuwe ventilator niet binnen, en ze is leeg. Wie ze oproept, denkt dat het gelukt is, terwijl er niets gebeurde.

Richting. Haal de methode weg uit de interface en uit de klasse. Vervangen 6 toont waarmee je het wél doet.

Je vraag: horen de gegevens uit punt 5 bij vervangen?

Nee. Je vermoeden is begrijpelijk: in 02_02 staat vervangen (punt 8) vlak na de testen (punt 7), en beide gebruiken een ventilator van 92 mm. Maar ze testen iets anders:

Punt	Waarover het gaat	Wat je test
03_02, punt 1	welke endpoints na je refactoring hetzelfde moeten antwoorden; vervangen is er één van	het antwoord van elk endpoint, vóór en na je refactoring
02_02, punt 8	wat vervangen precies doet	404, 400 of 201, en: mislukt het, dan is er niets veranderd
03_02, punt 5	of je lijst blijft bestaan tussen twee verzoeken	een POST van 92 mm, twee keer dezelfde DELETE, een PUT gevolgd door een GET
03_02, punt 6	de historiek	vervangen schrijft er twee wijzigingen in

De drie controles uit punt 5 gaan over toevoegen, verwijderen en bijwerken. Ze staan daar omdat ze pas iets bewijzen als je lijst onthoudt wat er gebeurde: een DELETE die de tweede keer 404 geeft, kan enkel als de eerste DELETE bewaard bleef.

Wat je wél uit punt 5 meeneemt naar vervangen, is de manier van testen. Je doet een verzoek, en kijkt met een tweede verzoek na wat er veranderd is. Zo werkt ook de test van vervangen in de nieuwe opgave, punt 4.

Stappenplan: vervangen

Wat er moet gebeuren, staat in de nieuwe opgave, punt 4, bij *Vervangen: één beslissing, twee acties*. Hieronder staat per stap waar je werkt, wat er in je code al klaarstaat, en hoe je weet dat de stap klopt. Werk in deze volgorde.

Vervangen 1 · Het endpoint en zijn route

Doel. Een PUT-endpoint op `api/ventilator/vervang/{id}`.

Nu. Regel 117 tot 121, in commentaar. De route begint met `/`.

Uitleg. De route op een methode komt achter de route van de controller, `api/[controller]`, behalve als ze met `/` begint. Dan telt ze vanaf het begin van het adres. Met jouw route zou je endpoint op `/vervang/{id}` liggen, zonder `api/ventilator` ervoor. PUT `api/ventilator/vervang/3` zou dan nog altijd een 404 zonder body geven.

Lees. Hoofdstuk 25, *Veelgemaakte fouten*.

Let op. Zodra je de commentaartekens weghaalt, moet elke weg door je methode eindigen met een `return`, anders bouwt je project niet: *not all code paths return a value*. Wil je na elke stap kunnen bouwen, gebruik dan tijdelijk de plaatshouder uit hoofdstuk 36, stap 3.

Klaar als Scalar onder *Ventilator* een PUT toont met het pad `/api/Ventilator/vervang/{id}`.

Vervangen 2 · Wat er binnenkomt

Doel. Het endpoint krijgt twee dingen: het Id uit de URL en de nieuwe ventilator uit de body.

Nu. Je methode heeft enkel een `int id`. De nieuwe ventilator komt nergens binnen.

Uitleg. Met `[ApiController]` beslist ASP.NET Core zelf waar een parameter vandaan komt: een eenvoudig type met dezelfde naam als de parameter in de route komt uit de URL, een object zoals een `Ventilator` uit de body. Je PUT voor bijwerken (regel 68) heeft precies die twee.

Lees. Hoofdstuk 25, *Waar een parameter vandaan komt*.

Klaar als het endpoint in Scalar een body heeft, met een voorbeeld van een ventilator.

Vervangen 3 · Het verzoek loggen

Doel. Als eerste regel een Information-bericht dat zegt welke ventilator vervangen wordt.

Nu. Nog niets. Doe het zoals in B1: vóór elke controle.

Klaar als elk verzoek naar dit endpoint, ook een geweigerd, een `info:`-regel in je terminal zet.

Vervangen 4 · De oude ventilator moet bestaan

Doel. Bestaat de ventilator uit de URL niet, dan 404 met *Kan ventilator met Id {id} niet vervangen, omdat deze niet gevonden is.*, en een Warning.

Nu. Je PUT voor bijwerken doet precies dit, op regel 70 tot 74, met een ander bericht.

Vraag je af. Waarom staat deze controle vóór de drie andere? Wie iets wil vervangen wat niet bestaat, krijgt altijd een 404, ook met een ongeldige body. Zo staat het in punt 1.

Klaar als PUT `api/ventilator/vervang/99` een 404 met het bericht geeft, ook met een body van 92 mm.

Vervangen 5 · De drie controles op de nieuwe ventilator

Doel. Dezelfde drie controles als bij toevoegen, met dezelfde redenen, en bij elke 400 een Warning met de reden.

Nu. Je POST heeft ze op regel 43 tot 59. Neem die als voorbeeld, niet je PUT: die heeft de fout uit A2.

Klaar als PUT `api/ventilator/vervang/2` met 92 mm een 400 met de reden geeft, en GET `api/ventilator/2` daarna nog altijd de oude ventilator toont.

Vervangen 6 · Uitvoeren: verwijderen en toevoegen

Doel. Pas nu verandert er iets: de oude ventilator gaat weg, de nieuwe komt erbij.

Nu. Je repository kan beide al: `Delete` en `Create`. Meer heb je niet nodig.

Vraag je af. Welk object geeft `Create` terug, en welke Id heeft het? Hou het bij: je hebt het nodig in Vervangen 7, en in Historiek 8.

En ook. Maakt het uit of je eerst verwijdert of eerst toevoegt, voor de Id van de nieuwe ventilator? Probeer het met de ventilator met de hoogste Id. Beide volgordes mogen; de opgave zegt er iets over onder de test in punt 4.

Klaar als na een geldige vervanging GET op de oude Id een 404 geeft, en de volledige lijst de nieuwe ventilator bevat.

Vervangen 7 · Het antwoord: 201 met het adres van de nieuwe

Doel. 201 Created met de nieuwe ventilator, en een header `Location` die naar de nieuwe wijst.

Nu. Je POST doet dit op regel 64.

Vraag je af. Welke Id hoort in het adres: die uit de URL, of die van de nieuwe ventilator? De oude bestaat niet meer: een adres ernaar zou een 404 geven.

Klaar als de header `Location` eindigt op `/api/Ventilator/` en de Id van de nieuwe ventilator. Scalar toont de headers van het antwoord bij *Headers*.

Doe daarna de test uit de nieuwe opgave, punt 4. De twee wijzigingen voor de historiek komen erbij in Historiek 8.

Stappenplan: de historiek

De nieuwe opgave deelt punt 6 op in vijf delen, 6a tot 6e. Hieronder staan ze in negen kleinere stappen, met bij elke stap wat er in je code al staat. Bouw je project na elke stap.

Historiek 1 · Het model (6a) ✓

`Models/Wijziging.cs` heeft de drie gegevens uit de tabel. Omdat `GlobalUsings.cs` de map Models in je hele project bekendmaakt (regel 3), kan je `Wijziging` overal gebruiken zonder extra `using`. Niets te doen.

Historiek 2 · Het contract (6b)

Doel. Twee methodes: een wijziging bewaren, en alle wijzigingen teruggeven.

Nu. Teruggeven staat erin ✓. Bewaren ontbreekt. `wijzigingById` vraagt de opgave niet.

Vraag je af. Wat krijgt de methode om te bewaren binnen, en wat geeft ze terug? De tabel in 6b zegt het: een wijziging zonder Id, en niets.

Over `wijzigingById`. In C# begint een methode met een hoofdletter. Visual Studio toont daarom drie grijze puntjes onder de naam: suggestie IDE1006, een naamregel. Je hebt de methode niet nodig: haal ze weg, uit de interface en uit de klasse.

Klaar als Visual Studio de naam van je klasse rood onderlijnt, met *does not implement interface member*. Dat is goed: de compiler bewaakt nu je contract. Met Ctrl+. maakt hij de ontbrekende methode voor je aan (hoofdstuk 36, stap 3).

Historiek 3 · Een lijst die blijft bestaan (6b)

Doel. Een lijst met wijzigingen die leeg begint en alle verzoeken overleeft.

Nu. Regel 5 is een gewoon veld, niet `static`.

Uitleg. Met `AddScoped` krijgt elk verzoek een nieuwe repository. Een gewoon veld hoort bij dat ene object: elk verzoek begint dan met een nieuwe, lege lijst, en je historiek is altijd leeg. Je loste het al op voor je ventilatoren, op regel 5 van `InMemoryVentilatorRepository`.

Lees. Hoofdstuk 24, en hoofdstuk 27, *Hoe lang een dienst leeft*.

Klaar als na Historiek 9 twee POST's na elkaar twee wijzigingen in de historiek zetten.

Historiek 4 · Alle wijzigingen teruggeven (6b)

Dit is A1. Sorteren is niet nodig: je lijst staat al in de juiste volgorde. **Klaar als** je project weer bouwt.

Historiek 5 · Een wijziging bewaren, met een eigen Id (6b)

Doel. De repository geeft de wijziging een Id, en voegt ze toe aan de lijst. De regel uit de opgave: 1 voor de allereerste, daarna telkens één meer dan de vorige.

Vraag je af. Je ventilatorrepository neemt het hoogste Id plus één. Wat doet dat op een lege lijst? Hoofdstuk 11, *Let op bij LINQ*, zegt het in één zin.

En ook. Wordt er in de historiek ooit iets verwijderd? Wat zegt het aantal wijzigingen dan over het Id van de volgende? Er zijn meer juiste manieren: kies er een die ook bij een lege lijst werkt.

Klaar als na Historiek 9 de eerste wijziging na een herstart Id 1 heeft.

Historiek 6 · Program.cs (6c)

Doel. ASP.NET Core weet welke klasse het moet aanleveren als iemand om een `IWijzigingRepository` vraagt.

Nu. Regel 13 en 14 registreren je twee andere repositories. De wijzigingsrepository staat er niet bij.

Lees. Hoofdstuk 27.

Klaar als je project bouwt. Je ziet nog niets veranderen: pas in Historiek 7 vraagt iemand erom.

Historiek 7 · De constructor van je controller (6c)

Doel. De controller krijgt de wijzigingsrepository binnen, en bewaart ze in een `private readonly` veld.

Nu. Regel 9 tot 16 doen dit al voor de ventilatorrepository en de logger. Er komt een derde parameter en een derde veld bij, op dezelfde manier.

Let op. Vergat je Historiek 6, dan bouwt je project wel, maar geeft élk verzoek naar je controller een 500. In je terminal staat *Unable to resolve service for type*, met de naam van je interface. De fout komt pas als ASP.NET Core je controller probeert te maken.

Klaar als GET `api/ventilator` nog altijd 200 geeft.

Historiek 8 · Een wijziging bewaren na elke geslaagde actie (6d)

Doel. Na elke geslaagde actie vraagt de controller de wijzigingsrepository om een wijziging te bewaren. Waar dat in jouw code komt:

Endpoint	Waar de wijziging komt	Actie	VentilatorId
POST	na `Create` (regel 61), vóór de `return` (regel 64)	`toegevoegd`	het Id van het object dat `Create` teruggaf
PUT, bijwerken	na `Update` (regel 95), vóór de `return` (regel 97)	`bijgewerkt`	het Id uit de URL
DELETE	na `Delete` (regel 111), vóór de `return` (regel 114)	`verwijderd`	het Id uit de URL
PUT, vervangen	na Vervangen 6, vóór de `return`	eerst `verwijderd`, dan `toegevoegd`	eerst het Id uit de URL, dan dat van de nieuwe ventilator

Waarom er dan geen spoor blijft. Elke 400 en 404 staat hoger in je methode, met een `return`. Een `return` verlaat de methode meteen: wat eronder staat, wordt dan nooit uitgevoerd. Staat je wijziging na de actie, dan bereikt een mislukt verzoek ze dus nooit. Je hoeft niets extra te controleren.

Een wijziging maken. Een nieuw `Wijziging`-object met een actie en een VentilatorId. Het Id vul je niet in: dat kiest de repository, zoals de client geen Id hoeft mee te sturen bij een POST.

Let op. Schrijf de acties letterlijk zoals in de tabel, in kleine letters. Het nakijkscript vergelijkt ze letterlijk.

Klaar als het testscenario hieronder klopt.

Historiek 9 · De historiek opvragen (6e)

Doel. GET `api/ventilator/historiek` geeft 200 met alle wijzigingen, of met `[]`.

Nu. Dit verzoek geeft 400: het komt bij GET `{id}` terecht, en van *historiek* kan ASP.NET Core geen `int` maken. Door `[ApiController]` krijg je dan automatisch een 400.

Uitleg. Zodra je een endpoint met de route `historiek` maakt, kiest ASP.NET Core dat. Passen er twee routes, dan gaat een vast stuk tekst voor op een parameter. `historiek` en `{id}` botsen dus niet. Twee parameters, zoals `{id}` en `{naam}`, zouden wel botsen (hoofdstuk 25, *Voorspel de uitkomst*).

Vraag je af. Wat geef je terug als de lijst leeg is? Een lege historiek is geen fout: 200 met `[]`, geen 404. Hoofdstuk 36, stap 10, legt uit waarom.

Klaar als GET `api/ventilator/historiek` na een herstart 200 met `[]` geeft, en er een `info:`-regel in je terminal staat.

Alles samen testen

Doorloop het testscenario uit de nieuwe opgave, punt 6. Heeft je startlijst de Id's 1 tot en met 4, dan ziet je historiek er na stap 6 zo uit:

id	actie	ventilatorId	Waarom
1	`toegevoegd`	5	stap 2: de Noctua krijgt Id 5
2	`bijgewerkt`	5	stap 4: dezelfde Noctua, het Id uit de URL
3	`verwijderd`	1	stap 5: de oude ventilator, het Id uit de URL
4	`toegevoegd`	6	stap 5: de Arctic, het Id dat de repository koos

Stap 3 en 6 voegen niets toe: een 400 en een 404 laten geen spoor na. Draai daarna het nakijkscript met `Nakijk_03_02`. Het doorloopt dit scenario en de regels uit punt 1, 3 en 5, en toont per stap wat je API antwoordt.

C. Opkuis, zonder invloed op het antwoord

Waar	Wat	Richting
controller, regel 18	`[HttpGet()]` met lege haakjes	`[HttpGet]` doet hetzelfde.
controller, regel 28, 62, 94 en 112	logberichten in het Engels, of half Engels; de rest van je log is Nederlands	Kies één taal. Neem het mee met B1.
controller, regel 62	een `$` voor een tekst zonder `{...}` erin	Het `$` doet daar niets.
controller, regel 28, 33, 72, 94, 107 en 112	logberichten met `$"..."`. Visual Studio toont daar mogelijk drie grijze puntjes: suggestie CA2254. De logger bewaart een waarde liever apart, met een plaatshouder in een vaste tekst. Zo kan je later op die waarde zoeken.	Werkt zoals het is. Wil je de suggestie weg, lees dan *CA2254* op Microsoft Learn.
beide repositories, regel 5	de lijst zelf wordt nooit vervangen, enkel aangevuld. Visual Studio kan IDE0044 voorstellen: *Make field readonly*.	`readonly` zegt: dit veld wijst altijd naar dezelfde lijst. Wat erin zit, mag nog veranderen. Combineer het met Historiek 3.